

Prompt:

Role

You are an expert Academic Editor and Senior Software Architect specializing in Systems Programming (Java and Rust) and Distributed Systems. Your goal is to refine a Master's Thesis draft into a professional Computer Science paper.

Task

Refine the provided text to improve its academic tone, technical precision, and logical flow. The thesis focuses on the modernization and reimplementing of a Virtual Resource Management (VRM) system from Java to Rust, including architectural extensions.

Constraints & Style Guidelines

1. **Tone:** Maintain a formal, objective, and analytical academic style. Use the passive voice where appropriate for technical descriptions, but keep active voice for system "actions."
2. **Terminology:** Use standard systems programming and scheduling terminology (e.g., "abstraction layer," "state synchronization," "deterministic scheduling," "resource orchestration").
3. **Clarity:** Ensure the relationship between components (e.g., Master ADC, Acl, and RMS Connectors) is logically sound.
4. **Correction:** Fix all grammatical errors, specifically focusing on:
 - Distinguishing between "then" (time) and "than" (comparison).
 - Subject-verb agreement.
 - Correct use of technical pluralization (e.g., "criteria" vs "criterion").
5. **Rust Context:** Since the project moved to Rust, subtly highlight concepts like safety, concurrency, or performance where the architecture implies it.
6. **Scope:** Do not add unnecessary text. Be professional but concise. Also do not use - or ; if it is possible. Additionally, do not replace acronyms.
7. **Return Type:** Latex format

Do not add new information.

Draft Input

The probe process in the VRM is different from other states because these states are deeply related to each other, and a successful probe reservation passes through all states. The probe process is initialised by the `\rstruct{VrmManager}`, which changes the state of a reservation from `\emph{Open}` to `\emph{ProbeAnswer}` and signals that for this reservation there is a probe reservation in the system requested. The potential outcomes of this request are `\emph{Rejected}` in case of a failure or `\emph{ReserAnswer}` in success.

In the probe process, all connected `\rstruct{Acl}`s are requests that return the so-called `\emph{ProbeReservation}`s, which are reservations that satisfy the requirements of the

requested reservation. The `\emph{ProbeReservation}`s are aggregated in a `\rstruct{ProbeReservations}` object. This object contains the original probe reservation and all `\emph{ProbeReservation}`s with their `\emph{AcId}`, from which this probe reservation originates. The difference between a probe reservation and a normal reservation is that probe reservations are not tracked by the `\rstruct{ReservationStore}`. These aggregated `\rstruct{ProbeReservations}` are returned to their requester, where the promotion process begins. The best probe reservation of the `\rstruct{ProbeReservation}` object is selected (e.g., earliest start time), and the reserve process for this reservation and the state are changed to `\emph{ReserveProbeReservation}`. The reservation is directly submitted to this component via the `\emph{AcId}`. If the reserve request is successful, the state is changed to `\emph{ReserveAnswer}`, otherwise the promoted probe reservation is deleted and the next best reservation is prompted.

Code-Snippet

Output:

The probe process within the VRM architecture is distinct from other operational states because these phases are highly interdependent, requiring a successful probe operation to traverse every intermediate state. This process is instantiated by the `\rstruct{VrmManager}`, which transitions the reservation state from `\emph{Open}` to `\emph{ProbeAnswer}`. This transition indicates that a probe request has been dispatched within the system. The potential outcomes of this operation are `\emph{Rejected}` upon failure or `\emph{ReserAnswer}` following a successful execution.

During the probe phase, the system queries all connected `\rstruct{Ac}` components to retrieve `\emph{ProbeReservation}` objects that satisfy the specific requirements of the request. These objects are aggregated into a `\rstruct{ProbeReservations}` container. This container encapsulates the primary probe request and all retrieved `\emph{ProbeReservation}` instances alongside their respective `\emph{AcId}` to ensure origin traceability. A significant architectural distinction is that `\emph{ProbeReservation}` instances are not persisted within the `\rstruct{ReservationStore}`, which prioritizes memory safety and performance by avoiding unnecessary state tracking.

These aggregated `\rstruct{ProbeReservations}` are returned to the requester to initiate the promotion phase. The system selects the optimal candidate from the `\rstruct{ProbeReservations}` collection based on selection criteria such as the earliest start time. Consequently, the reservation state transitions to `\emph{ReserveProbeReservation}` and the request is dispatched directly to the component identified by the `\emph{AcId}`. If the reserve request succeeds, the state transitions to `\emph{ReserveAnswer}`. In the event of a

failure, the system discards the promoted candidate and evaluates the next optimal reservation for promotion.